

turknet

2026-27 GÜZ DÖNEMİ · SEÇMELİ LİSANS DERSİ

# Güvenli Kodlama ve Programlama

Hafta 1 · Tanışma ve temel kavramlar



Can Öztaş

[canoztas.github.io/courses/secure-coding](https://canoztas.github.io/courses/secure-coding)

## PROGRAM

# Bu ders hangi kapsamda veriliyor?



T.C. Sanayi ve Teknoloji  
Bakanlığı

turknet

Turknet

NEV | İSTANBUL  
NİŞANTAŞI  
UNIVERSITY

İstanbul Nişantaşı  
Üniversitesi



Karamanoğlu  
Mehmetbey  
Üniversitesi

BUGÜN

# Bu derste ne yapacağız

- |    |                          |                                                                                  |
|----|--------------------------|----------------------------------------------------------------------------------|
| 01 | Tanışma                  | Ben kimim, siz kimsiniz, bu sınıfta kim ne biliyor.                              |
| 02 | Dersin çerçevesi         | Haftalık plan, notlandırma, dönem ödevi, etik kurallar.                          |
| 03 | Siber güvenlik temelleri | Saldırgan, saldırı, savunma; zafiyet, exploit, risk, güven sınırı, CVE/CWE/CVSS. |
| 04 | İlk kod                  | Beş satırlık bir fonksiyonu birlikte kuralım.                                    |

TANIŞMA

Hadi tanışalım

R. Can Öztaş



## TANIŞMA

# Şimdi sizi tanıyalım

### 1. Siber güvenliğe merakınız ve ilginiz

Nereden geliyor? CTF, haber, bir olay, bir hoca, sadece merak?

### 2. Programlama ve geliştirme geçmişiniz

Hangi diller, ne tür projeler; okul ödevi mi, gerçek kullanıcıya giden kod mu?

### 3. Bu dersten beklentileriniz

Ne öğrenmiş olarak çıkmak istiyorsunuz? Kariyer, CTF, staj, araştırma?

# Bu ders ne, ne değil

## Bu ders

- Kodun nasıl kırıldığını anlamak
- Kök nedeni (root cause) koda kadar takip etmek
- Savunmayı tasarıma ve koda gömmek
- Dilden ve çerçeveden bağımsız ilkeler

## Bu ders değil

- Sızma testi (pentest) sertifika kursu
- Tek bir aracın kullanım eğitimi
- Tek bir dile bağlı bir ders
- Ezberlenecek bir açık listesi

# Çalışan kod, güvenli kod demek değildir.

Derleyici ve testler “kod ne yapmalı?” sorusunu sorar. Saldırgan “kod başka ne yapabilir?” sorusunu sorar. Bu ders, ikinci sorunun dersi.



# Dönem sonunda ne yapabileceksiniz

## Kırabilmek

Yaygın zafiyet sınıflarını kontrollü bir ortamda tetikleyebilmek.

## Okuyabilmek

Bir kod parçasına bakıp güven sınırlarını ve riskli noktaları çıkarabilmek.

## Düzeltebilmek

Belirtiyi değil kök nedeni kapatan düzeltmeyi yazabilmek.

## Doğrulamak

SAST ve DAST çıktısını yorumlayıp yanlış pozitif (false positive) ayıracabilmek.

## Savunabilmek

Bir tasarım kararını güvenlik gerekçesiyle ekibe anlatabilmek.

## YÖNTEM

# Önce saldırgan, sonra savunma

### 1 • Kır

Zafiyeti kendi laboratuvarınızda gerçekten tetikleyin. Slaytta okumak yetmez.



### 2 • Anla

Hangi varsayım yanlıştı? Kök neden hangi satırda, hangi tasarım kararında?



### 3 • Savun

Aynı sınıftaki tüm hataları kapatan yapısal düzeltmeyi yazın.

## PLAN

# Haftalık plan • 1-8

| Hafta | Konu                                                                   |
|-------|------------------------------------------------------------------------|
| 1     | Giriş, güvenli tasarım ilkeleri ve tehdit modelleme (threat modeling)  |
| 2     | Enjeksiyon (injection) ve güvenilmeyen girdinin işlenmesi              |
| 3     | Tarayıcı güvenlik modeli, XSS ve CSRF                                  |
| 4     | Uygulamalı kriptografi (applied cryptography)                          |
| 5     | Kimlik doğrulama, parolalar ve oturum yönetimi (session management)    |
| 6     | Yetkilendirme, erişim kontrolü (access control) ve API güvenliği       |
| 7     | C/C++ ve bellek güvenliği (memory safety)                              |
| 8     | Güvenli yapılandırma, loglama, tedarik zinciri (supply chain) ve CI/CD |

## PLAN

# Haftalık plan • 9-15

| Hafta | Konu                                                                  |
|-------|-----------------------------------------------------------------------|
| 9     | Statik ve dinamik analiz (SAST/DAST) ile zafiyet tespiti ve triyaj    |
| 10    | Yapay zekâ destekli geliřtirmenin (AI-assisted development) güvenliđi |
| 11    | LLM uygulamalarının güvenliđi                                         |
| 12    | Proje sunumları                                                       |
| 13    | Proje sunumları                                                       |
| 14    | Genel tekrar                                                          |
| 15    | Final sınavı                                                          |

ÇERÇEVE

# Notlandırma

**%60**

## Final sınavı

Dönem sonu; kod okuma ve gerekçelendirme ağırlıklı.

**%20**

## Ara sınav (vize)

Üniversitelerin vize haftasında; dönemin ilk yarısının konuları, kod okuma ve düzeltme.

**%20**

## Dönem projesi

Zafiyet araştırması ve sunumu: gerçek bir zafiyetin kök nedeni, sömürü yolu, yama eleştirisi; rapor ve sunum.

Sınav tarihleri üniversitelerin akademik takvimine göre ders sayfasında duyurulur.

# Dönem ödevi: zafiyet araştırması ve sunumu

Ekipler, haftalık konularla eşleşen gerçek ve yaması yayımlanmış bir zafiyeti (CVE ya da bilinen bir olay) listeden seçer, kaynağına inip sınıfa anlatır.

## Zafiyetli kod

Hatalı kod parçası ve kök neden.

## Sömürü yolu

Saldırgan bu hatayı hangi adımlarla kullanıyor?

## Yama eleştirisi

Yama (patch) kök nedeni kapatıyor mu, belirtiyi mi?

## Dersle bağ

Hangi hafta, hangi pratik bunu engellerdi?

Ekip büyüklüğü sınıf mevcuduna göre belirlenecek. Sunum sonrası soru-cevap **bireysel** notlandırılır.

## KURALLAR

# Etik ve yasal çerçeve

### İzin yoksa test yok

Sahibi olmadığınız hiçbir sisteme, “sadece denedim” diye bile dokunmayın.

### Kendi laboratuvarınız

Tüm pratikler yerel, izole ve kasıtlı zafiyetli ortamlarda yapılır.

### Yasal zemin

Türk Ceza Kanunu 243-245: bilişim sistemine izinsiz girmek, sistemi engellemek ve veriyi bozmak suçtur.

### Sorumlu bildirim

Gerçek bir zafiyet bulursanız önce üreticiye bildirin (responsible disclosure); yayımlamadan önce benimle konuşun.

## BÖLÜM 2

# Siber güvenlik temelleri



# Saldırgan kim, ne ister?



## Meraklı / script kiddie

Hazır araçlarla dener; hedef seçmez, açık olanı bulur. Otomatik tarama sizi de bulur.



## Hacktivist

Mesaj vermek ister: web sayfası değiştirme, veri sızdırma, hizmet durdurma.



## Siber suç örgütü

Para ister: fidye yazılımı, kart verisi, hesap ele geçirme. Bugün en yaygın tehdit.



## İçeriden tehdit / insider

Zaten yetkisi var; kötü niyet ya da ihmal. Erişim kontrolünün asıl sınavı.



## Devlet destekli / APT

Sabırlı, kaynaklı, hedefli. Casusluk, sabotaj, tedarik zinciri saldırıları.



## Araştırmacı / bug bounty

İzinle test eder, sorumlu bildirir. Bu dersin sonunda sizin olacağınız yer.

Ortak nokta: hepsi **en ucuz yolu** arar. Saldırganın maliyetini yükseltmek, savunmanın işidir.

# Şapkalar: kim hangi tarafta?



## Beyaz şapka / white hat

İzinle test eder, bulduğunu sahibine bildirir. Pentester, güvenlik araştırmacısı, bug bounty avcısı. Bu dersin hedefi.



## Gri şapka / gray hat

İzin almadan bulur, çoğu zaman bildirir; bazen ücret ister ya da ifşa eder. Niyet iyi olsa da hukuken riskli bölge.



## Siyah şapka / black hat

Kâr, zarar ya da güç için izinsiz sızar. Suç örgütleri, fidye çeteleri, veri hırsızları.



## Yeşil şapka

Acemi ama öğrenmeye hevesli; topluluklarda, CTF'lerde büyür.



## Mavi şapka

Dışarıdan davetle test eden (Microsoft BlueHat) ya da intikam amaçlı tek seferlik saldırgan.



## Kırmızı şapka

Siyah şapkaları kendi yöntemleriyle avlayan “kanunsuz” savunmacı. Etik değil, sadece renk değişmiş.

Rengi belirleyen teknik değil, **izin ve niyet**. Aynı araç, aynı yöntem; şapkayı yalnızca “kimin izniyle, ne için?” sorusu değiştirir.

## TEMELLER

# Bir saldırının anatomisi



01

### Keşif

Hedefi tanı: alan adları, çalışanlar, sürümler, açık portlar.



02

### İlk erişim

Ölçümler, sızdırılmış parola, açık servisteki zafiyet.



03

### Çalıştırma

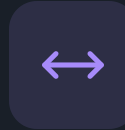
Kod çalıştır: exploit, makro, zararlı yazılım, web shell.



04

### Kalıcılık ve yetki yükseltme

Sistemde kal; kullanıcıdan yöneticiye çık.



05

### Yanal hareket

Bir makineden ağın geri kalanına yayıl.



06

### Hedef

Veri sızdır, şifrele, boz ya da sessizce izle.

Literatürdeki adları: **Cyber Kill Chain** (Lockheed Martin) ve **MITRE ATT&CK** taktik matrisi. Zinciri herhangi bir halkada kırmak saldırıyı durdurur; güvenli kod, 02 ve 03'ü pahalılaştırır.

## TEMELLER

# Savunma nasıl kurulur?



### İnsan ve süreç

Farkındalık, parola politikası, olay müdahale planı.



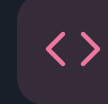
### Ağ

Segmentasyon, güvenlik duvarı, TLS, sıfır güven (zero trust).



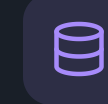
### Uç nokta

Güncel sistem, EDR, en az yetkiyle çalışan servisler.



### Uygulama

Girdi doğrulama, güvenli kütüphane, kod incelemesi, SAST/DAST.



### Veri

Şifreleme, yedek, erişim kaydı, en az veri toplama.



### Katmanlı savunma / defense in depth

Tek bir kontrol her zaman yenilir; katmanlar birbirini tamamlar.



### İhlali varsay / assume breach

Soru “girerler mi?” değil, “girdiklerinde ne olur?”



### Asimetri

Saldırganın bir kez haklı olması yeter; savunmacı her seferinde haklı olmalı.

Bu dersin yeri: **Uygulama** katmanı. Diğer katmanlar kodun hatasını kapatabilir ama kodun kendisi güvenli değilse en pahalı yerden kırılırsınız.

## TEMELLER

# CTF nedir?

**Capture The Flag:** kasıtlı olarak zafiyetli sistemlerde, yasal ve kontrollü bir ortamda saklanan “bayrağı” bulma yarışması. İki biçim: **Jeopardy** (kategorilere ayrılmış bağımsız sorular) ve **Attack–Defense** (takımlar hem kendi servisini savunur hem rakibinkine saldırır).

### WEB

XSS, SQLi, kimlik doğrulama hataları. Bu dersin ana sahası.

### PWN / BINARY

Bellek hataları, taşma, ROP. 7. haftanın konusu.

### REVERSE

Kaynak kodu olmayan bir programı anlamak.

### CRYPTO

Yanlış kullanılmış şifrelemeyi kırmak.

### FORENSICS / OSINT

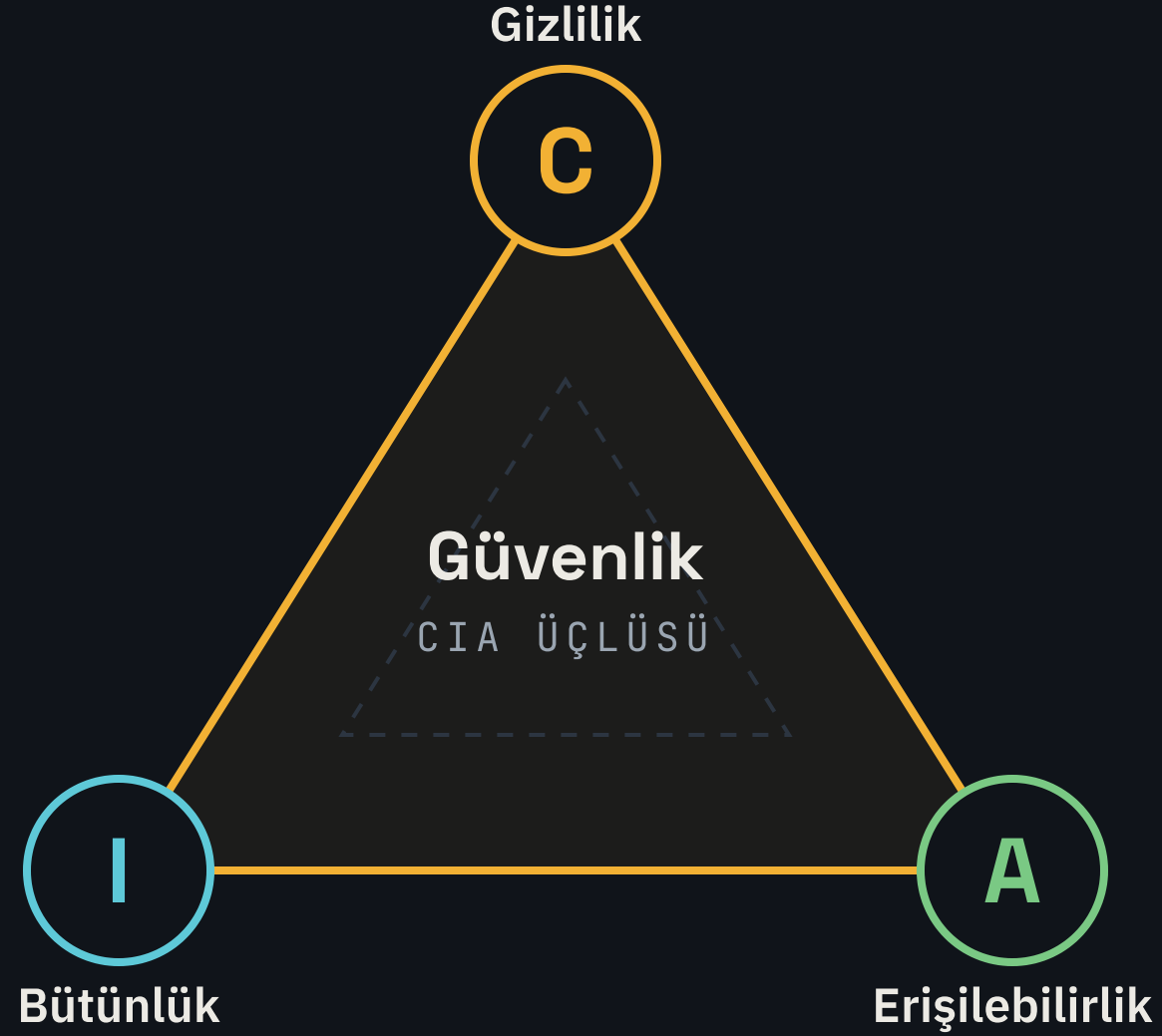
Disk, ağ trafiği, log; açık kaynaklardan iz sürmek.

### MISC

Hepsinden biraz; yaratıcılık ödüllendirilir.

Başlangıç için: **picoCTF**, **PortSwigger Web Security Academy**, **TryHackMe**, **Hack The Box**; takvim için **ctftime.org**. Bu derste öğrendiğiniz her şeyi güvenle deneyeceğiniz yer burası.

# Güvenlik neyi korur?



## Gizlilik / Confidentiality

Veriyi yalnızca yetkili taraflar görebilir. Kırıldığında: veri sızıntısı.

## Bütünlük / Integrity

Veri yalnızca yetkili yollarla değişir. Kırıldığında: bakiye, rol ya da fiyat değişir.

## Erişilebilirlik / Availability

Sistem ihtiyaç duyulduğunda çalışır. Kırıldığında: hizmet durur.

Üçlüye iki soru eşlik eder: **kim olduğunu** nasıl biliyoruz (kimlik doğrulama / authentication), **neye hakkı olduğunu** nasıl biliyoruz (yetkilendirme / authorization).

## KAVRAM

# Hata, zafiyet, sömürü

| Terim                   | Ne demek                                           | Örnek                                 |
|-------------------------|----------------------------------------------------|---------------------------------------|
| Hata / bug              | Beklenen davranıştan sapma.                        | Sayfa 500 hatası veriyor.             |
| Zafiyet / vulnerability | Güvenlik sınırını aşmaya izin veren hata.          | Girdi doğrudan SQL sorgusuna giriyor. |
| Sömürü / exploit        | Zafiyeti tetikleyen somut girdi ya da kod.         | id=1 OR 1=1                           |
| Yük / payload           | Sömürü başarılı olunca çalışan iş.                 | Kullanıcı tablosunu dışarı aktar.     |
| 0-gün / zero-day        | Yaması olmayan zafiyet. n-day: yaması çıkmış olan. | Yama sonrası hâlâ açık sistemler.     |

Her zafiyet bir hatadır; her hata zafiyet değildir. Fark: güvenlik sınırını aşıp aşmadığı.

# Risk nasıl hesaplanır

$$\text{Risk} \approx \text{Tehdit} \times \text{Zafiyet} \times \text{Etki}$$

## Tehdit / threat

Kötüye kullanma niyeti ve kapasitesi olan taraf.

## Zafiyet / vulnerability

Bu tarafın kullanabileceği zayıflık.

## Etki / impact

Kırılırsa ne kaybediyoruz?

## Saldırı yüzeyi / attack surface

Dışarıdan ulaşılabilen giriş noktalarının toplamı.

Çarpanlardan biri sıfırsa risk de sıfırdır. Güvenli kodlama ortadaki çarpanı düşürür; mimari kararlar saldırı yüzeyini küçültür.

## KAVRAM

# Güven sınırı / trust boundary

### GÜVENİLMİYEN / UNTRUSTED

#### Tarayıcı, mobil uygulama, API istemcisi

Kontrolü sizde değil. İstemcide yaptığınız her doğrulama isteğe bağlıdır.

### GÜVENİLEN / TRUSTED

#### Sunucu, iş mantığı, veritabanı

Kararların gerçekten alındığı yer. Doğrulama burada yapılır.

Sınırı geçen her şey girdidir: form alanı, başlık, çerez, dosya adı, başka bir servisin JSON'u, kendi veritabanınızdan gelen eski kayıt. **Kanıtlanana kadar hepsi güvenilmezdir.**

## KAVRAM

# Zafiyetin ortak dili

| Kısaltma     | Ne işe yarar                                                   | Örnek                       |
|--------------|----------------------------------------------------------------|-----------------------------|
| CVE          | Belirli bir ürünlerdeki belirli bir zafiyetin kimlik numarası. | CVE-2021-44228              |
| CWE          | Zafiyetin türü, sınıfı.                                        | CWE-89: SQL enjeksiyonu     |
| CVSS         | Şiddeti 0-10 arası puanlayan ölçek.                            | 9.8 (kritik)                |
| OWASP Top 10 | En yaygın web risk kategorileri listesi.                       | Erişim kontrolü zafiyetleri |
| KEV          | Sömürüldüğü doğrulanmış zafiyetler kataloğu.                   | Yama önceliklendirmesi      |

CVSS puanı sizin bağlamınızı bilmez: kritik puanlı bir zafiyet sizin mimarinizde ulaşamaz olabilir, orta puanlı biri kritik olabilir.

# Beş satır, iki soru

```
@app.route("/profil")
def profil():
    uid = request.args.get("id")
    q = "SELECT ad, eposta FROM kullanicilar WHERE id = " + uid
    return db.execute(q).fetchone()
```

## Soru 1

Bu kod ne yapmalı? Testler bu soruyu sorar ve kod geçer.

## Soru 2

Bu kod başka ne yapabilir? Saldırgan yalnızca bunu sorar.

# Aynı kod, saldırganın gözüyle

```
GET /profil?id=1
GET /profil?id=1 OR 1=1
GET /profil?id=1 UNION SELECT parola_ozeti, NULL FROM kullanicilar
```

## Kök neden / root cause

Veri ile kod aynı kanalda birleşti. Sorgu metni, kullanıcı girdisiyle birlikte kuruluyor.

## Savunma / prepared statement

Sorgunun iskeleti sabit kalır, veri parametre olarak ayrı gider. Kaçış karakteri eklemek değil, kanalları ayırmak.

Bu bir yazım hatası değil, tasarım hatası. Aynı kalıbı 2. haftada komut satırında, dosya yollarında ve şablon motorlarında tekrar göreceğiz.

## İLKELER

# Saltzer ve Schroeder, 1975

### En az yetki / least privilege

Her bileşen işini yapmaya yetecek en dar yetkiyle çalışsın.

### Güvenli varsayılan / fail-safe defaults

Varsayılan karar reddetmek olsun; erişim açıkça verilsin.

### Tam aracılık / complete mediation

Her erişim, her seferinde kontrol edilsin.

### Açık tasarım / open design

Güvenlik, tasarımın gizliliğine değil anahtara dayansın.

### Ekonomik mekanizma / economy of mechanism

Basit tut: incelenemeyen karmaşıklık güvenli değildir.

### Kabul edilebilirlik / psychological acceptability

Kullanıcıyı zorlayan güvenlik, atlatılan güvenliktir.

## İLKELER

# Güvenlik ne zaman düşünülür?

### Tasarım

Tehdit modelleme

### Kodlama

Güvenli kalıplar

### İnceleme

Kod incelemesi

### Test / CI

SAST, DAST, bağımlılık  
taraması

### İşletme

Loglama, izleme, yama

Cevap: hepsinde. Ama düzeltme sağa gittikçe pahalılaşır. Literatürdeki adı **shift-left**: kararı mümkün olan en erken aşamaya çekmek.

## KAYNAKLAR

# Dönem boyunca başvuracağınız kaynaklar

### STANDARTLAR VE REHBERLER

- ▶ OWASP Top 10 (2025)
- ▶ OWASP Top 10 for LLM Applications
- ▶ OWASP Application Security Verification Standard (ASVS)
- ▶ OWASP Cheat Sheet Series
- ▶ SEI CERT Secure Coding Standards

### ÖNERİLEN KİTAPLAR

- ▶ Mathias Payer, *Software Security: Principles, Policies, and Protection* (SS3P) — [nebelwelt.net/SS3P](https://nebelwelt.net/SS3P)
- ▶ Robert C. Seacord, *Secure Coding in C and C++*, 2. Baskı, Addison-Wesley, 2013
- ▶ Dan Bergh Johnsson, Daniel Deogun & Daniel Sawano, *Secure by Design*, Manning, 2019
- ▶ Adam Shostack, *Threat Modeling: Designing for Security*, Wiley, 2014
- ▶ Dafydd Stuttard & Marcus Pinto, *The Web Application Hacker's Handbook*, 2. Baskı, Wiley, 2011
- ▶ Michael Howard & David LeBlanc, *Writing Secure Code*, 2. Baskı, Microsoft Press, 2003

### EK KAYNAKLAR

- ▶ OpenSSF, Developing Secure Software (LFD121) — [training.linuxfoundation.org](https://training.linuxfoundation.org)
- ▶ Stanford CS 253 Web Security — [web.stanford.edu/class/cs253](https://web.stanford.edu/class/cs253)

Tam liste ve bağlantılar ders sayfasında: [canoztas.github.io/courses/secure-coding](https://canoztas.github.io/courses/secure-coding)

KAPANIŞ

# Haftaya kadar şunlara bakın

## Burp Suite

Community Edition'ı kurun, bir web sitesine proxy üzerinden bakın. İstek/yanıtı görmeyi öğrenin.

## Web güvenliği temelleri

OWASP Top 10 listesine göz gezdirin; HTTP, çerez, oturum kavramlarını hatırlayın.

## Stanford CS 253

Web Security ders konularına bakın: tarayıcı güvenlik modeli, XSS, CSRF.

Haftaya: **tehdit modelleme** — bir sistemi saldırganın gözünden çizmek.

SORU • CEVAP

# Sorular?

---

